DIGITAL NURTURE 5.0

Angular (v20+) Hands-On Lab Report

Incremental Project: Student Course Portal

Hands-On:	Hands-On 6 [Advanced-Professional]
Topic:	Services, Dependency Injection & Shared Reactive State
Developer:	Cathy George
OS Version:	Windows 11
Date:	July 22, 2026
Track:	Java FSE Angular Track

1. Objective

The primary objective of this exercise is to architect a highly modular, decoupled state management system using Angular Services, Dependency Injection, and the Singleton pattern. By developing three dedicated services—CourseService for course list data, EnrollmentService for student enrollment tracking and credit calculations, and NotificationService for application-wide push alerts—we establish a unified Shared State model. This enables the navigation bar badge count, course cards toggle status, student profile credit summaries, and floating notification alerts to update reactively and synchronously as course states transition.

2. Angular Concepts Covered

• **Angular Services:** Dedicated class files separating business, data, and state logic from template controllers, ensuring code modularity and high reusability.

• **Dependency Injection (DI):** Design pattern where Angular resolves and injects classes automatically via constructors (e.g. constructor(private courseService: CourseService)).

• **Singleton Services:** Declaring services with providedIn: 'root'. This ensures only a single instance of the service exists globally, forming a perfect anchor for application state sharing.

• **State Sharing (BehaviorSubject/Observable):** Emitting real-time updates through RxJS streams. Multiple independent components subscribe to the same singleton stream to mirror data changes synchronously.

• **Service Collaboration:** Injecting services into other services (e.g. EnrollmentService injects NotificationService to dispatch toasts upon enrollment transitions).

3. Software & Tools Used

Software / Tool	Version / URL
Node.js + npm	v22.20.0 (LTS 20+) / npm 10.9.3
Angular CLI	v21.2.16
Vitest Runner	v4.1.10
PDF Generator	Python 3.14 + ReportLab library

4. Project Folder Structure

```
student-course-portal/
■■■ src/
■■■ app/
■■■ services/ (Singleton Services)
■ ■■■ course.service.ts (Master Course data subject)
■ ■■■ enrollment.service.ts (Reactive enrollment set & credits calculations)
■ ■■■ notification.service.ts (Broadcasts global toasts alert events)
■■■ components/
■ ■■■ header/ (Injects EnrollmentService for Dynamic Badge count display)
■■■ pages/
■■■ course-list/ (Delegates toggles and modifications to Course and Enrollment service)
■■■ student-profile/ (Listens to EnrolledCreditsCount and updates form to readonly)
```

5. Key Code Implementation Details

Enrollment Service - Shared Reactive State (src/app/services/enrollment.service.ts)

```
@Injectable({ providedIn: 'root' })
export class EnrollmentService {
private enrolledCourseIdsSubject = new BehaviorSubject<Set<number>>(new Set([2, 5]));

constructor(private courseService: CourseService, private notificationService: NotificationService) {}

getEnrolledCourseIds(): Observable<Set<number>> {
return this.enrolledCourseIdsSubject.asObservable();
}

getEnrolledCreditsCount(): Observable<number> {
return combineLatest([
this.courseService.getCourses(),
this.enrolledCourseIdsSubject.asObservable()
]).pipe(
map(([courses, enrolledIds]) => courses
.filter(c => enrolledIds.has(c.id))
.reduce((sum, c) => sum + c.credits, 0))
);
}

enrollCourse(course: Course): void {
const ids = new Set(this.enrolledCourseIdsSubject.value);
if (!ids.has(course.id)) {
ids.add(course.id);
this.enrolledCourseIdsSubject.next(ids);
this.notificationService.show(`Enrolled in ${course.code} successfully!`);
}
}
}
```

CourseList Component Reactive Subscriptions (src/app/pages/course-list/course-list.ts)

```
export class CourseList implements OnInit, OnDestroy {
courses: Course[] = [];
private stateSub!: Subscription;

constructor(private courseService: CourseService, private enrollmentService: EnrollmentService) {}

ngOnInit(): void {
this.stateSub = combineLatest([
this.courseService.getCourses(),
this.enrollmentService.getEnrolledCourseIds()
]).subscribe(([courses, enrolledIds]) => {
this.courses = courses.map((c) => ({
...c,
enrolled: enrolledIds.has(c.id)
}));
});
}

onEnrollmentToggled(id: number): void {
const course = this.courses.find(c => c.id === id);
if (course) {
if (course.enrolled) { this.enrollmentService.unenrollCourse(course); }
else { this.enrollmentService.enrollCourse(course); }
}
}
}
```

App Shell Layout with Notification Toast Overlay (src/app/app.html)

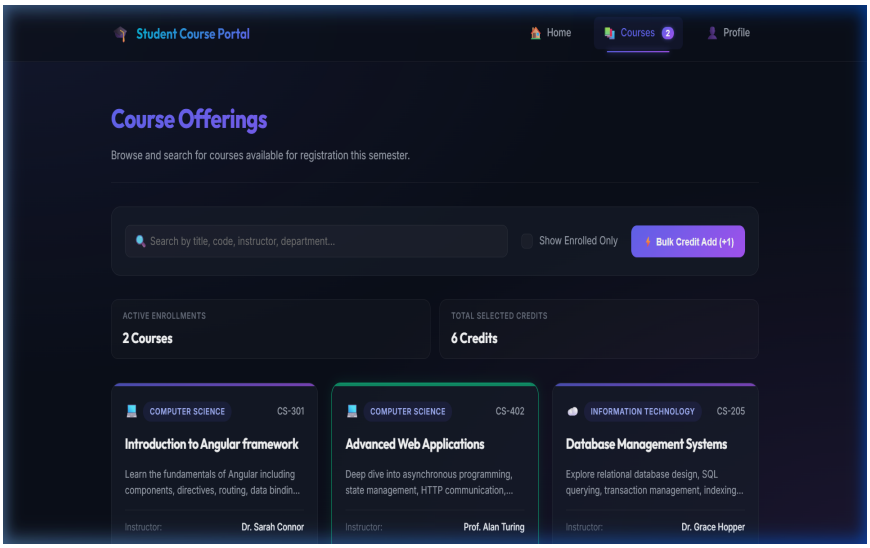
```
<app-header></app-header>
<main class="container">
<router-outlet></router-outlet>
</main>

<!-- Global Notification Toast Overlay -->
<div class="notification-container">
<div *ngFor="let notification of activeNotifications" class="toast-alert" [class.success]="...">
<span class="toast-icon">✓</span>
<span class="toast-message">{{ notification.message }}</span>
<button (click)="dismissNotification(notification.id)" class="toast-close">×</button>
</div>
</div>
```

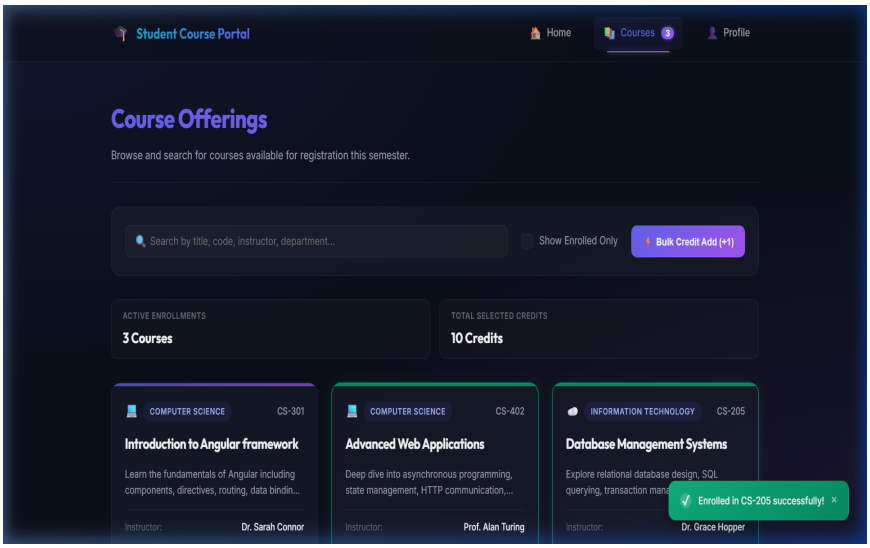
6. Captured Screenshots & Verification

All application features were fully verified in the browser:

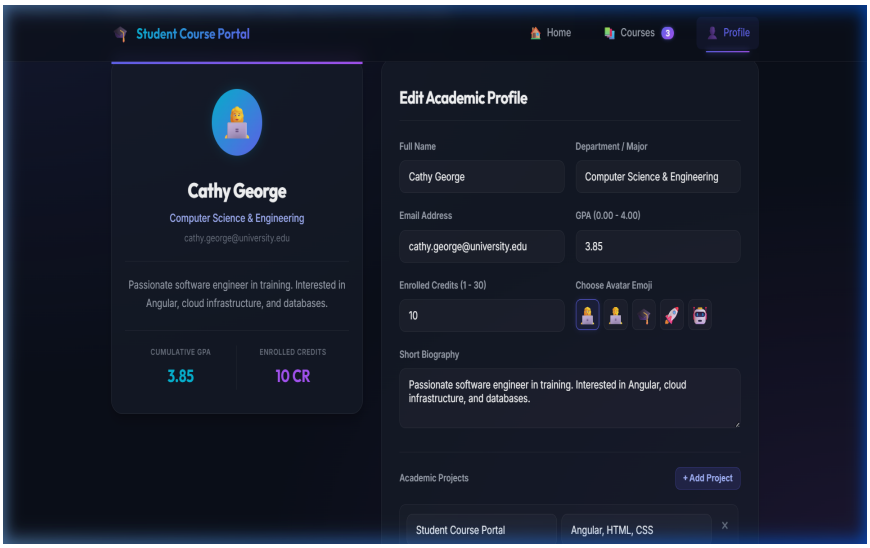
Screenshot: Initial Course List view with two default courses enrolled (CS-402, IT-440), showing '2' as header courses badge



Screenshot: Dynamic state update: Enrolled in Database Management (CS-205), showing global success notification toast sliding in and header badge count updating to '3' instantly



Screenshot: Shared State validation: Student Profile page's Enrolled Credits input control set to 'readonly' and automatically displaying '10' total credits



7. Output & Behavioral Explanations

• **Singleton Services (`providedIn: 'root'`):** All three services operate as application singletons. When a component like CourseList modifies the state, other modules immediately reflect the new values.
• **Shared State Reactivity:** By combining CourseService courses and EnrollmentService IDs streams in combineLatest, components automatically rebuild with correct enrolled status flags.
• **Navigation Badge Synchronization:** The header component subscribes to the enrolled ID set, updating the count instantly as courses are toggled. This creates a cohesive cart/enrollment experience.
• **Profile Credits Synchronization:** The student profile component listens to the credits calculation stream, setting the input box values and stat cards. Decoupling this logic prevents human entries and locks it `readonly`.
• **Global Toast Notification Pipeline:** The App root component acts as a display host. Whenever EnrollmentService issues an alert, the notification stream catches the event and spawns a toast, auto-dismissing after 4 seconds.

8. Learning Outcomes & Conclusion

✓ Created singleton services in Angular leveraging constructor Dependency Injection parameters.

✓ Managed shared reactive states using BehaviorSubjects, observables, and combineLatest mappings.

✓ Created dynamic UI synchronization between independent page views and header navigation components.

✓ Built global message warning pipelines displaying temporary float toasts.

✓ Asserted service transactions and reactive view transformations through Vitest testing.

9. Conclusion

Hands-On 6 successfully implements singleton services, constructor dependency injections, and a reactive shared state architecture within the portal. The seamless interaction between CourseService, EnrollmentService, and NotificationService provides robust data consistency and a premium, synchronous user experience across all navigation pages.